

# proctor\_

Agentic coding bills by the token, and the expensive mistakes are invisible in an invoice: a cache that quietly stopped working, a session re-reading its own history, the same instructions pasted into thirty conversations. proctor reads the transcripts you already have and puts a dollar figure on each one.

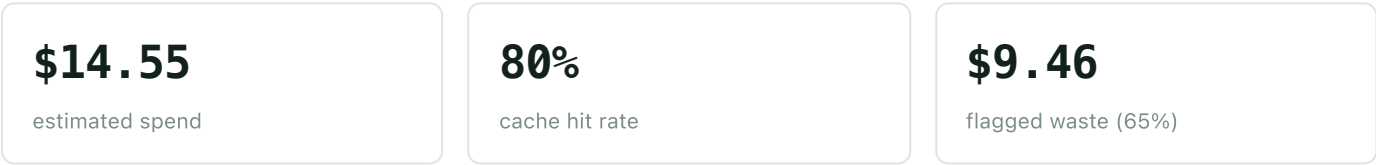
---

## THE PROBLEM

### An invoice tells you the total. It doesn't tell you which habit caused it.

Claude Code writes a full transcript of every session to disk, including the exact token usage the API reported for each turn. That is enough to reconstruct where the money went — but nothing reads it. Teams end up guessing, or reacting only when a monthly bill crosses a threshold.

The costly patterns are structural rather than occasional. Prompt caching is a prefix match, so a single interpolated timestamp can silently drop the hit rate across every session. Cache reads bill at 10% of base input, which makes a broken cache roughly a **10x** multiplier on the affected context.



Figures from the demonstration corpus used throughout this document — 13 sessions, 102 API turns.

Run one command against transcripts you already have

```
proctor -- 13 sessions, 102 API turns  (last 90 days)

estimated spend                $14.55
input (uncached)              1,603,413 tok
cache writes                   239,000 tok
cache reads                    7,278,721 tok  (hit rate 80%)
output                         83,422 tok
flagged waste                  $9.46  (65% of spend)

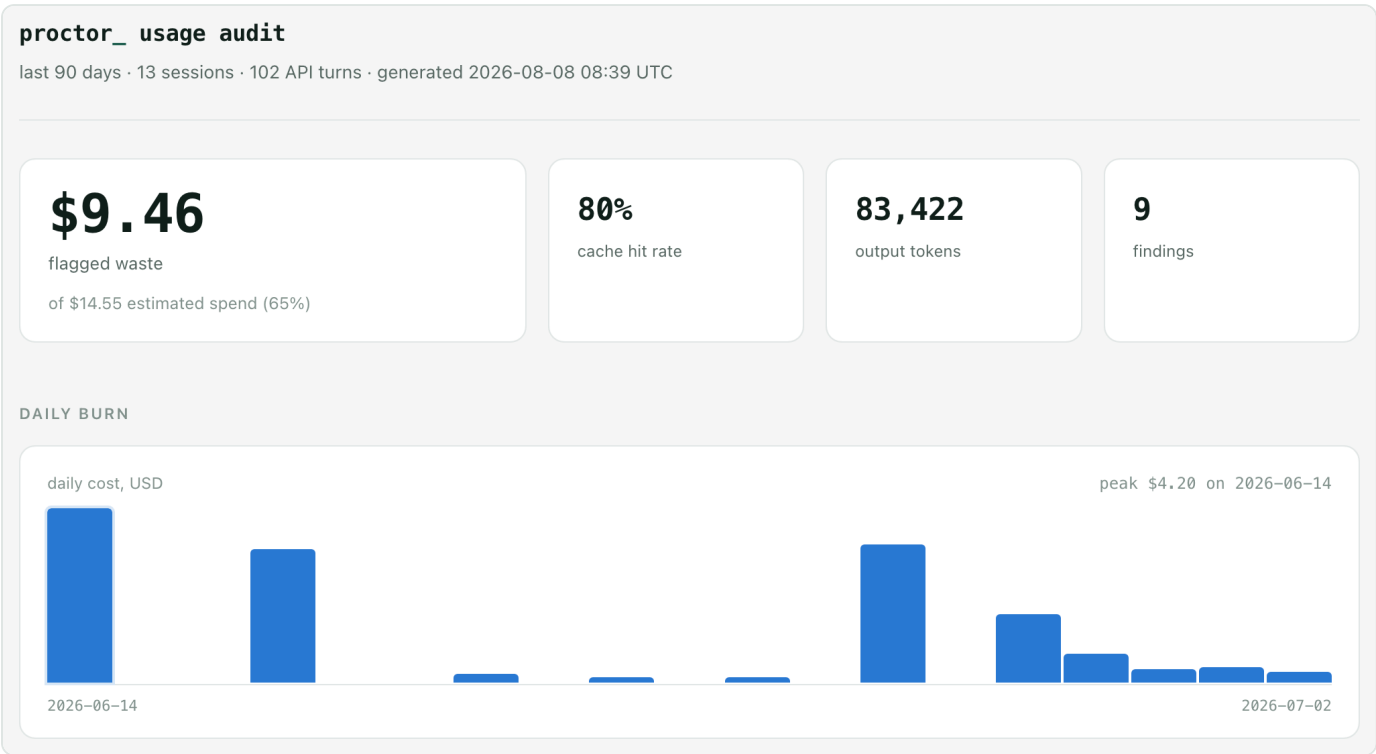
daily burn

2026-06-14 → 2026-07-02    peak $4.20/day

spend by model
  claude-opus-5                $12.08    83%
  claude-fable-5               $1.51     10%
  claude-sonnet-5              $0.96      7%
```

Terminal output. Demonstration corpus — every figure below comes from the same synthetic dataset, generated by `examples/generate_demo.py`, never from real transcripts.

Spend, cache health, and a ranked worklist



The HTML report leads with flagged waste against total spend, then the daily burn. A single self-contained file that opens offline.

FINDINGS

Ranked by dollars, so you know where to start

Each finding names the session, explains the mechanism, and estimates what it cost. The bar under each amount encodes its share of the largest finding, so the worklist is scannable before it is read.

| FINDINGS, RANKED BY ESTIMATED WASTE                                                                                                                                                                                                                                                        |                      |                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------|
| \$2.67                                                                                                                                                                                                                                                                                     | COLD CONTEXT         | a3f21c88 · /srv/acme/billing  |
| 10 turn(s) reprocessed 658,880 uncached input tokens mid-session. Cache reads cost 10% of base input, so this context was paid for at full price a second time. Usually caused by editing the system prompt, changing the tool set, or letting the 5-minute cache TTL lapse between turns. |                      |                               |
| \$2.21                                                                                                                                                                                                                                                                                     | COLD CONTEXT         | e5c07b90 · /srv/acme/ledger   |
| 9 turn(s) reprocessed 545,045 uncached input tokens mid-session. Cache reads cost 10% of base input, so this context was paid for at full price a second time. Usually caused by editing the system prompt, changing the tool set, or letting the 5-minute cache TTL lapse between turns.  |                      |                               |
| \$1.43                                                                                                                                                                                                                                                                                     | SUBAGENT BURN        | e5c07b90 · /srv/acme/ledger   |
| Subagents consumed \$2.86 of this session's \$3.20 (89%). Each subagent starts cold and re-derives context the parent already had. Worth it for genuinely parallel work; expensive for anything sequential.                                                                                |                      |                               |
| \$1.19                                                                                                                                                                                                                                                                                     | BALLOONED SESSION    | b7d40e12 · /srv/acme/ledger   |
| Context grew to 359,645 tokens over 26 turns while producing only 4,898 output tokens (1.4%). The back half of the session spent ~\$1.98 re-reading accumulated context. Compact or restart before a session snowballs.                                                                    |                      |                               |
| \$1.06                                                                                                                                                                                                                                                                                     | MODEL MISMATCH       | f2b93d07 · /srv/acme/billing  |
| claude-fable-5 produced only 180 output tokens for \$1.51. The same token volume on Sonnet would be ~\$0.45. Check whether the task needed the top tier — model choice is yours to make, and this is only a cost comparison, not a quality one.                                            |                      |                               |
| \$0.28                                                                                                                                                                                                                                                                                     | MODEL MISMATCH       | 0c6ad4e1 · /srv/acme/workers  |
| claude-opus-5 produced only 1,900 output tokens for \$0.70. The same token volume on Sonnet would be ~\$0.42. Check whether the task needed the top tier — model choice is yours to make, and this is only a cost comparison, not a quality one.                                           |                      |                               |
| \$0.26                                                                                                                                                                                                                                                                                     | REDUNDANT TOOL CALLS | c1e88a34 · /srv/acme/payments |
| Read(file_path=/srv/payments/client.py) ran 6 times with identical arguments, each pulling back ~8,197 tokens. The agent re-read work it already had in context.                                                                                                                           |                      |                               |
| \$0.21                                                                                                                                                                                                                                                                                     | COLD CONTEXT         | 0c6ad4e1 · /srv/acme/workers  |
| 1 turn(s) reprocessed 52,000 uncached input tokens mid-session. Cache reads cost 10% of base input, so this context was paid for at full price a second time. Usually caused by editing the system prompt, changing the tool set, or letting the 5-minute cache TTL lapse between turns.   |                      |                               |
| \$0.17                                                                                                                                                                                                                                                                                     | OVERSIZED PROMPT     | 0c6ad4e1 · /srv/acme/workers  |
| ~33,652-token prompt against a median of ~11. That text stays in context for every turn that follows it. "Here are the logs: 2026-06-14T02:00:11Z worker-0 WARN queue depth 900 exceeded soft limit, deferring batch 0 2..."                                                               |                      |                               |

Findings from the demonstration corpus.

Which sessions, which projects, which models

| Top sessions by cost |                    |       |              |                  |        |
|----------------------|--------------------|-------|--------------|------------------|--------|
| Session              | Project            | Turns | Peak context | Cache hit        | Cost   |
| a3f21c88             | /srv/acme/billing  | 11    | 124,000      | <div></div> 0%   | \$4.20 |
| b7d40e12             | /srv/acme/ledger   | 26    | 359,645      | <div></div> 100% | \$3.21 |
| e5c07b90             | /srv/acme/ledger   | 11    | 66,443       | <div></div> 4%   | \$3.20 |
| f2b93d07             | /srv/acme/billing  | 1     | 148,000      | <div></div> 0%   | \$1.51 |
| 0c6ad4e1             | /srv/acme/workers  | 3     | 104,000      | <div></div> 33%  | \$0.70 |
| 33cc44dd             | /srv/acme/workers  | 14    | 36,000       | <div></div> 90%  | \$0.38 |
| 11aa22bb             | /srv/acme/platform | 12    | 36,000       | <div></div> 87%  | \$0.32 |
| 55ee66ff             | /srv/acme/payments | 10    | 36,000       | <div></div> 85%  | \$0.26 |
| c1e88a34             | /srv/acme/payments | 6     | 48,798       | <div></div> 99%  | \$0.22 |
| d0a11f55             | /srv/acme/platform | 2     | 18,000       | <div></div> 34%  | \$0.14 |

| Spend by model  |                |         |
|-----------------|----------------|---------|
| Model           | Share of spend | Cost    |
| claude-opus-5   | <div></div>    | \$12.08 |
| claude-fable-5  | <div></div>    | \$1.51  |
| claude-sonnet-5 | <div></div>    | \$0.96  |

Cache hit rate carries a status colour beside the number, so a failing session is visible without reading every row.

Three things this changes

| USE             | WHAT IT GIVES YOU                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Find the leak   | Cache misses are the single most expensive failure and the least visible. proctor names the session, the token count, and the likely cause — an edited system prompt, a changed tool set, a lapsed TTL. |
| Set a budget    | <code>--fail-over</code> exits non-zero when flagged waste crosses a threshold, so a nightly job fails loudly before a bill does. Exit 1 is over budget; exit 2 means the audit could not run.          |
| Attribute spend | Per-project and per-model breakdowns turn "our Claude bill went up" into a specific session on a specific day.                                                                                          |

FROM THIS RUN

| FIGURE | MEANING                                                  |
|--------|----------------------------------------------------------|
| 80%    | cache hit rate across 102 API turns                      |
| \$9.46 | flagged waste, 65% of \$14.55 estimated spend            |
| \$2.67 | largest single finding — cache miss in /srv/acme/billing |

**On the numbers.** Token counts are exact — they come from the usage object the API returns per turn. Costs are estimates at list prices and will not match an invoice; they ignore negotiated rates and subscription plans. Waste figures use deliberately conservative recovery factors, and findings overlap, so flagged waste can exceed total spend. Rank by individual finding rather than reading the total as a recoverable sum.

PRACTICALITIES

|          |                                                                            |
|----------|----------------------------------------------------------------------------|
| Install  | <code>pip install proctor-audit</code>                                     |
| Requires | Python 3.9+. No runtime dependencies.                                      |
| Data     | Runs locally. No network calls unless you ask for the organization report. |
| Privacy  | Reports quote your prompts, so treat JSON and HTML output as private.      |